System zapisów na zajęcia

Etap 2: architektura, prototypy i struktury danych

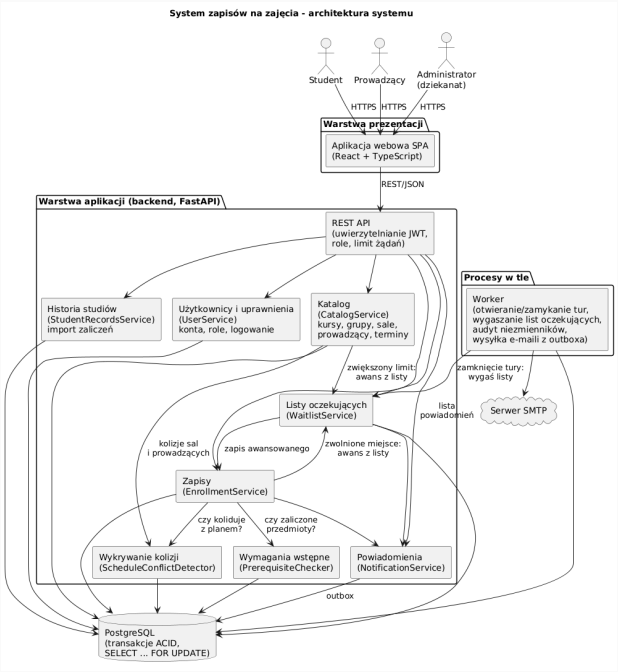
Kamil Kręcigłowa Bartosz Królik Krzysztof Krówka

21 października 2026

Inżynieria Oprogramowania, zima 2026/2027

Pokrycie wymagań

Wymaganie	Przypadki	Tabele (ER)	Prototyp
ograniczona liczba miejsc	UC2, UC13	`section.capacity`, `enrolled_count`, `waitlist_limit`	scen. 1, 5
katalog i harmonogram	UC1, UC11, UC12	`course`, `section`, `lecturer`, `meeting`, `room`, `term`, `enrollment_round`	scen. 6
różne rozmiary grup	UC12, UC13	`section.capacity`, `section.kind`, `room.seats`	scen. 1, 4, 5
wymagania wstępne	UC3, UC11, UC15	`course_prerequisite`, `completed_course`	scen. 2, 6
kolizje w harmonogramie	UC4, UC12	`meeting` (dzień, godziny, parzystość)	scen. 3
listy oczekujących	UC5, UC7, UC9, UC18	`waitlist_entry`, `notification`	scen. 4, 5, 6


System zapisów na zajęcia - architektura systemu
Student
Prowadzący
Administrator (dziekanat)
HTTPS
HTTPS
HTTPS
Warstwa prezentacji
Aplikacja webowa SPA (React + TypeScript)
REST/JSON
Warstwa aplikacji (backend, FastAPI)
REST API (uwierzytelnianie JWT, role, limit żądań)
Historia studiów (StudentRecordsService) import zaliczeń
Użytkownicy i uprawnienia (UserService) konta, role, logowanie
Katalog (CatalogService) kursy, grupy, sale, prowadzący, terminy
Procesy w tle
Worker (otwieranie/zamykanie tur, wygaszanie list oczekujących, audyt nieźmienników, wysyłka e-maili z outboxa)
zwiększony limit: awans z listy
zamknięcie tury: wygaś listy
Listy oczekujących (WaitlistService)
lista powiadomień
Serwer SMTP
kolizje sal i prowadzących
zapis awansowanego
zwolnione miejsce: awans z listy
Zapisy (EnrollmentService)
czy koliduje z planem?
czy zaliczone przedmioty?
Wykrywanie kolizji (ScheduleConflictDetector)
Wymagania wstępne (PrerequisiteChecker)
Powiadomienia (NotificationService)
outbox
PostgreSQL (transakcje ACID, SELECT ... FOR UPDATE)

Architektura i stos technologiczny

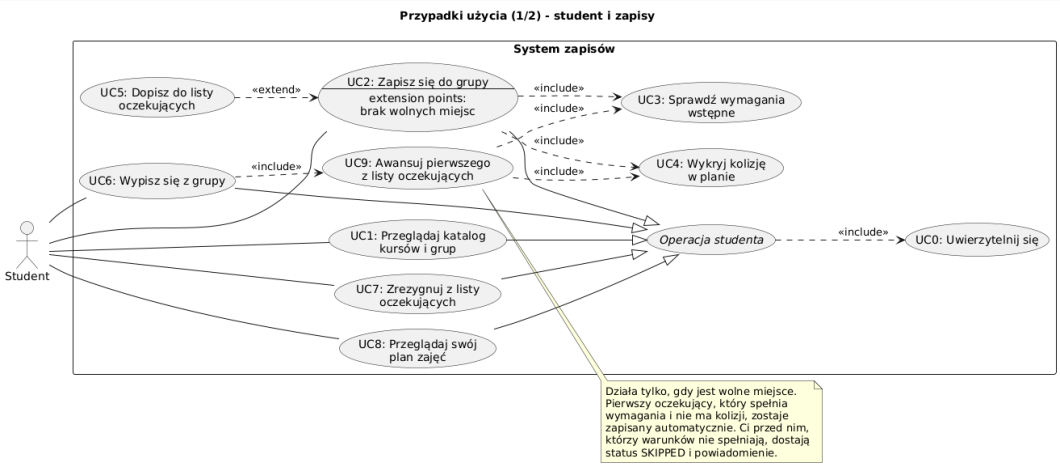
- **SPA** (React + TypeScript) dla studenta, prowadzącego, dziekanatu
- **REST API** (Python, FastAPI): JWT, role, limit żądań
- `EnrollmentService` – *jedyny* moduł zmieniający liczbę zajętych miejsc
- reguły: wymagania wstępne, kolizje; `WaitlistService`; outbox powiadomień
- **Worker**: tury zapisów, wygaszanie list, e-maile
- **PostgreSQL 16**: ACID, blokady wierszy, indeksy częściowe

Główne wyzwanie

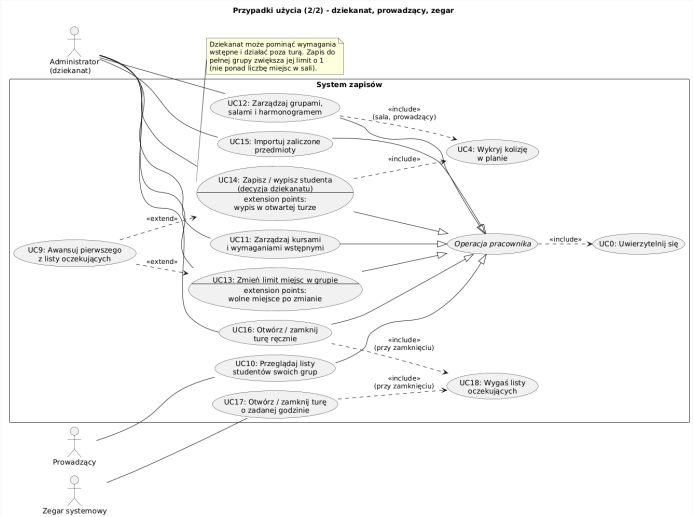
Otwarcie tury: setki studentów naraz walczą o te same miejsca.

- zapis do grupy serializuje blokada **wiersza grupy**
- zapisy jednego studenta – blokada **wiersza studenta**
- różni studenci w różnych grupach – równolegle
- outbox i tury w bazie $\Rightarrow$ bez osobnego brokera kolejek

Przypadki użycia (1/2): student



Przypadki użycia (2/2): dziekanat, prowadzący, zegar



Kluczowe scenariusze: zapis i awans z listy

UC2: Zapisz się do grupy

1. Blokada studenta, potem grupy.
2. Wymagania wstępne (UC3), kolizje (UC4), jedna grupa danego typu na przedmiot.
3. Wolne miejsce $\Rightarrow$ zapis, licznik $+1$.
4. Brak miejsc $\Rightarrow$ **UC5** (`<<extend>>`): koniec listy oczekujących.

Każda odmowa podaje przyczynę (który przedmiot, z jaką grupą).

UC9: Awans z listy

- w **tej samej transakcji**, która zwolniła miejsce
- kolejni oczekujący sprawdzani **ponownie**; niespełniający $\Rightarrow$ `SKIPPED` + powiadomienie
- bez wolnego miejsca nikogo nie pomijamy
- zamknięcie tury $\Rightarrow$ listy wygasają

Niezmiennik
Nikt nie czeka, gdy w grupie są wolne miejsca.


app_user
PK id bigserial
UQ email varchar(255)
password_hash varchar(255)
full_name varchar(200)
role varchar(16)
created_at timestamptz
lecturer
PK FK user_id bigint
title varchar(50)
department varchar(100)
section_lecturer
PK FK section_id bigint
PK FK lecturer_id bigint
meeting
PK id bigserial
FK section_id bigint
FK room_id bigint
weekday smallint
start_time time
end_time time
week_parity varchar(4)
section
PK id bigserial
FK course_id bigint
FK term_id bigint
code varchar(10)
kind varchar(16)
capacity smallint
enrolled_count smallint
waitlist_limit smallint
enrollment
PK id bigserial
FK student_id bigint
FK section_id bigint
status varchar(16)
source varchar(16)
enrolled_at timestamptz
dropped_at timestamptz
room
PK id bigserial
building varchar(20)
number varchar(20)
seats smallint
notification
PK id bigserial
FK user_id bigint
type varchar(32)
payload jsonb
created_at timestamptz
sent_at timestamptz
student
PK FK user_id bigint
UQ index_no varchar(10)
program varchar(100)
semester smallint
waitlist_entry
PK id bigserial
FK student_id bigint
FK section_id bigint
status varchar(16)
created_at timestamptz
resolved_at timestamptz
skip_reason varchar(255)
FK enrollment_id bigint
term
PK id bigserial
UQ code varchar(10)
starts_on date
ends_on date
enrollment_round
PK id bigserial
FK term_id bigint
name varchar(50)
opens_at timestamptz
closes_at timestamptz
status varchar(16)
course
PK id bigserial
UQ code varchar(20)
name varchar(200)
ects smallint
description text
course_prerequisite
PK FK course_id bigint
PK FK required_course_id bigint
completed_course
PK FK student_id bigint
PK FK course_id bigint
PK FK term_id bigint
grade numeric(2,1)
passed boolean
PK klucz główny
FK klucz obcy (kolor = tabela docelowa)
UQ wartość unikalna
kursywa – NULL dozwolony

Scenariusz 1: 30 studentów, 10 miejsc

Transakcja zapisu (`READ COMMITTED`):

```
BEGIN;
SELECT 1 FROM student
 WHERE user_id = :s FOR UPDATE;
SELECT ... FROM section
 WHERE id = :g FOR UPDATE;
-- tura, wymagania, kolizje
UPDATE section
   SET enrolled_count = enrolled_count + 1
 WHERE id = :g
   AND enrolled_count < capacity;
INSERT INTO enrollment ...
COMMIT;
```

Wynik: 10 zapisanych, 20 na liście.

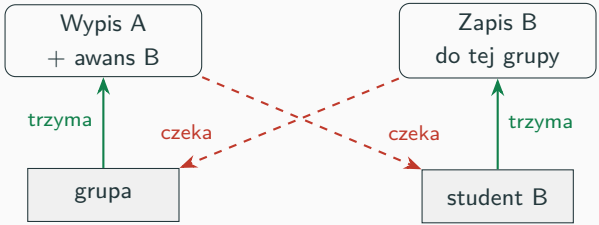
30 transakcji do jednej grupy:

- blokadę grupy ma jedna, reszta czeka,
- każda następna widzi licznik po `COMMIT` poprzedniej,
- jedenasta dostaje 0 wierszy z `UPDATE` i trafia na listę oczekujących.

Blokada studenta: jego dwa równoległe zapisy nie sprawdzą kolizji jednocześnie.

Bez blokad prototyp zapisał 30 osób przy liczniku równym 1.

Blokady w PostgreSQL: zakleszczenie przy awansie



Zapis blokuje studenta, potem grupę. Awans: grupę, potem studenta B.

Przy zwykłym `FOR UPDATE` PostgreSQL po `deadlock_timeout` (1 s) przerywa jedną z transakcji błędem 40P01 (sprawdzone na PostgreSQL 16).

Awans blokuje B bez czekania:

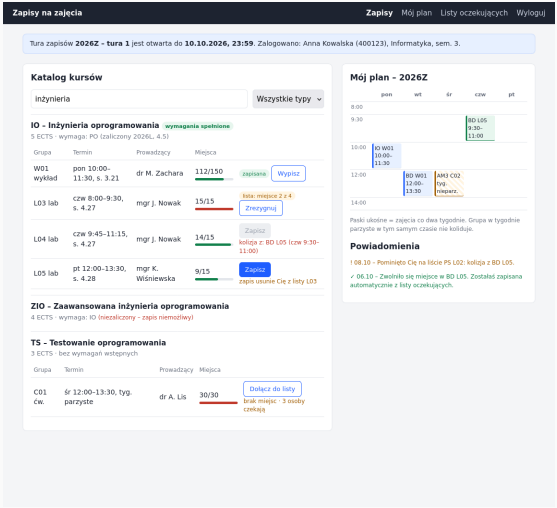
```
SELECT 1 FROM student
 WHERE user_id = :b
   FOR UPDATE NOWAIT;
```

Zajęty B: od razu błąd 55P03, wypis się wycofuje i jest ponawiany.

Awans zostaje w tej samej transakcji co zwolnienie miejsca, więc nikt spoza listy go nie zajmie, a B nie traci kolejki.

40P01 i 40001 obsługujemy tak samo.

Mock-up: ekran zapisów studenta



- katalog z zajętością miejsc
- kurs zablokowany przez niezaliczony przedmiot
- „Zapisz" zablokowane przez kolizję, z podaniem grupy
- pozycja na liście, „Zrezygnuj", „Dołącz do listy"
- ostrzeżenie: zapis usunie z listy innej grupy
- plan tygodnia z zajęciami co dwa tygodnie
- powiadomienia o awansie i pominięciu

Podsumowanie

Gotowe

- architektura i stos technologiczny
- przypadki użycia UC0–UC18
- diagram ER + `schemat.sql` (PostgreSQL 16)
- prototyp 6 scenariuszy ryzyka
- mock-up ekranu zapisów

Etap 3 (28.X)

- diagramy sekwencji
- diagram klas z metodami
- uprawnienia ról, obsługa błędów
- test blokad i wydajności

Dziękujemy!

Pytania?